

UNIVERSITAS SEBELAS MARET - SEKOLAH VOKASI
PROGRAM STUDI D3 TEKNIK INFORMATIKA
MODUL PRAKTIKUM PRAKTIK PEMROGRAMAN DRONT END — PERTEMUAN 5

BAB 5: Framework Modern UI & Ekosistem React (React 19, Vue 3, & Svelte 5)

Topik: Analisis Komparatif Framework UI, Automasi React Compiler pada React 19, Composition API & Vapor Mode Vue 3, serta Runes Reactivity Svelte 5

1. Deskripsi Singkat Mata Kuliah

Mata kuliah Pemrograman Front End Engineering pada Program Studi D3 Teknik Informatika Sekolah Vokasi Universitas Sebelas Maret (UNS) membekali mahasiswa dengan kompetensi rekayasa antarmuka web modern berbasis komponen (Component-Driven Development). Pada era pengembangan web terdistribusi dan cerdas saat ini, penguasaan kerangka kerja UI bukan sekadar menghafal sintaksis pustaka, melainkan memahami paradigma dasar reaktivitas, strategi kompilasi, serta optimasi performa pada level browser dan server.

Pertemuan ke-5 ini berfokus pada eksplorasi dan perbandingan arsitektural dari tiga kerangka kerja UI modern terpopuler di industri: React 19 (sebagai standar industri utama yang dilengkapi React Compiler), Vue 3 (dengan Composition API dan inovasi Vapor Mode), serta Svelte 5 (pionir compile-time reactivity berarsitektur Runes). Mahasiswa akan dipandu untuk memahami perbedaan mendalam antara Virtual DOM reconciliation, direct DOM updates, dan kompilasi murni, serta menerapkannya dalam pembuatan aplikasi yang efisien, interaktif, dan berstandar industri.

2. Tujuan Praktikum & Capaian Pembelajaran (CPMK)

Setelah menyelesaikan praktikum pada Bab 5 ini, mahasiswa D3 Teknik Informatika SV UNS diharapkan mampu:

- **Menganalisis Arsitektur & Performa UI Framework:** Memahami dan membandingkan secara kritis prinsip kerja React 19, Vue 3, dan Svelte 5 berdasarkan metrik ukuran bundle, eksekusi Virtual DOM vs Direct DOM, dan kepuasan pengembang.
- **Menguasai Fitur Unggulan React 19:** Mengimplementasikan aplikasi React 19 yang memanfaatkan keunggulan React Compiler untuk automasi memoization komponen tanpa perlu intervensi manual (useMemo/useCallback).
- **Menerapkan Composition API & Vapor Mode pada Vue 3:** Menyusun komponen reaktif Vue 3 berbasis Single File Component (.vue), mengelola state dinamis dengan ref/reactive, dan memahami paradigma Vapor Mode untuk pembaruan DOM langsung.

- **Mengimplementasikan Runes Reactivity pada Svelte 5:** Membangun aplikasi web ringan menggunakan Svelte 5 dengan memanfaatkan primitif reaktivitas Runes (`$state`, `$derived`, `$effect`) tanpa bergantung pada runtime Virtual DOM.
- **Menerjemahkan Spesifikasi SRS ke Komponen UI:** Mengidentifikasi dan menerjemahkan kebutuhan fungsional dari dokumen Software Requirements Specification (SRS) proyek kelompok/individu menjadi arsitektur komponen UI yang terstruktur dan scalable.

3. Dasar Teori Komprehensif & Valid

A. Lanskap & Statistik Adopsi Framework UI Modern (2026)

Pemilihan kerangka kerja (framework) antarmuka pengguna pada ekosistem JavaScript modern tidak lagi hanya didasarkan pada popularitas semata, melainkan evaluasi praktis terhadap kedalaman ekosistem, ketersediaan tenaga kerja di industri, kecepatan pengembangan tim, serta biaya pemeliharaan jangka panjang (SVAR UI, 2026). Berdasarkan data survei industri ekosistem JavaScript dan laporan unduhan NPM tahunan (SVAR UI, 2026; InfoQ, 2026), React secara konsisten mempertahankan posisinya sebagai pemimpin pasar dengan volume adopsi masif (~162.5 juta unduhan NPM tahunan dan tingkat penggunaan sekitar 83.6%–85% di antara para pengembang). Vue.js menempati posisi kedua secara global dengan tingkat penggunaan ~52% dan ~14.1 juta unduhan NPM tahunan, sementara Svelte bertindak sebagai penantang utama dengan peningkatan pertumbuhan pesat (~5.2 juta unduhan NPM) serta memimpin indikator kepuasan dan minat pengembang (63% Developer Interest dan 71% Positivity).

Metrik Evaluasi	React 19	Vue.js 3	Svelte 5
Unduhan NPM Tahunan	~162.5 Juta / tahun	~14.1 Juta / tahun	~5.2 Juta / tahun
Tingkat Penggunaan (State of JS)	~83.6% – 85% (Dominan)	~52% (Stabil & Luas)	~27% (Pertumbuhan Pesat)
Developer Sentiment	Sangat Tinggi & Stabil	Stabilitas & Kepuasan Tinggi	#1 Developer Interest (63%) & Positivity (71%)
Tata Kelola / Governance	React Foundation (Linux Foundation)	Independen & Komunitas (Evan You)	Komunitas & Dimentori Vercel

B. React 19: Evolusi React Compiler & Automasi Memoization

Selama bertahun-tahun, salah satu tantangan terbesar dalam pengembangan aplikasi React adalah manajemen re-render komponen yang tidak efisien. Pengembang secara manual harus menyisipkan hook optimasi seperti `useMemo`, `useCallback`, dan `React.memo` untuk mencegah komputasi ulang atau re-render komponen anak yang tidak perlu (SVAR UI, 2026; Egnworks, 2026). Pendekatan manual ini tidak

hanya menambah boilerplate code, tetapi juga rentan terhadap kesalahan pengembang (human error) seperti missing dependency array.

Peluncuran React 19 memperkenalkan React Compiler (sebelumnya dikenal sebagai Auto-Memoizing Compiler). React Compiler bekerja pada tingkat kompilasi (build-time) dengan menganalisis pohon komponen dan secara otomatis memvisualisasikan serta menerapkan memoization pada nilai dan fungsi yang tidak berubah secara dinamis (SVAR UI, 2026). Hal ini menghilangkan kebutuhan intervensi manual `useMemo/useCallback`, menurunkan beban eksekusi JavaScript di browser, dan memastikan aplikasi React berjalan pada performa puncak secara otomatis.

KEUNGGULAN GOVERNANCE REACT 19

React 19 secara resmi mengalihkan tata kelola intellectual property dan hak ciptanya di bawah naungan React Foundation yang dikelola oleh Linux Foundation bersama sponsor platinum seperti Meta, Microsoft, Amazon, Vercel, dan Huawei. Hal ini memberikan jaminan kepastian dan stabilitas jangka panjang bagi organisasi level enterprise (SVAR UI, 2026).

C. Vue 3: Composition API & Inovasi Vapor Mode

Vue 3 mempertahankan posisinya sebagai kerangka kerja yang sangat intuitif dan fleksibel melalui struktur Single File Component (.vue) yang menggabungkan template HTML, skrip logika JavaScript, dan scoped CSS dalam satu berkas terorganisir (SVAR UI, 2026). Pengenalan Composition API memberikan kebebasan bagi pengembang untuk mengelompokkan logika aplikasi berdasarkan fitur fungsional alih-alih pemisahan berorientasi opsi (Options API).

Guna mengatasi overhead rekonsiliasi Virtual DOM pada aplikasi berskala besar, Vue 3 memperkenalkan Vapor Mode — sebuah strategi kompilasi opt-in yang sepenuhnya mengabaikan Virtual DOM. Vapor Mode mengompilasi kode komponen Vue langsung menjadi instruksi pembaruan DOM berniat spesifik (direct DOM updates). Pendekatan ini membawa kecepatan eksekusi dan konsumsi memori Vue 3 ke dalam tingkat performa puncak yang setara dengan Svelte, tanpa merusak pengalaman pengembangan (Developer Experience) khas Vue (SVAR UI, 2026).

D. Svelte 5: Compile-Time Architecture & Runes Reactivity

Svelte mengambil pendekatan radikal dengan menggeser pekerjaan kerangka kerja dari runtime browser ke tahap kompilasi (build-time). Berbeda dengan React dan Vue tradisional yang menyertakan runtime Virtual DOM di dalam bundle browser, Svelte mengompilasi komponen menjadi kode JavaScript murni (vanilla JS) yang memanipulasi DOM secara langsung (SVAR UI, 2026). Hasilnya, rata-rata ukuran berkas gzipped aplikasi Svelte berada di kisaran ~31 KB, jauh lebih ringan dibandingkan aplikasi React yang dapat mencapai ~148 KB (SVAR UI, 2026).

Pada versi Svelte 5, sistem reaktivitas diperbarui secara mendasar melalui fitur yang disebut Runes. Runes adalah primitif reaktivitas eksplisit berbasis simbol khusus seperti `$state()`, `$derived()`, dan `$effect()`. Runes memungkinkan variabel reaktif dikelola secara deklaratif, transparan, dan dapat digunakan baik di dalam berkas komponen (.svelte) maupun di luar komponen (.svelte.js/.svelte.ts) tanpa keterbatasan ketergantungan scope (SVAR UI, 2026).

E. Matriks Perbandingan Arsitektur UI Framework 2026

Dimensi Arsitektur	React 19	Vue 3 (Composition / Vapor)	Svelte 5 (Runes)
Kurva Pembelajaran	Moderat – Tinggi (JSX, Hooks, Ekosistem Luas)	Rendah – Moderat (Template HTML, SFC, v-model)	Paling Rendah (Mendekati Vanilla HTML/JS, Runes)
Mekanisme Reaktivitas	State Hooks & React Compiler Automasi	Proxy-based Reactivity (ref/reactive)	Compile-Time Primitives (\$state, \$derived)
Render Engine	Virtual DOM + Auto-Memoization (Compiler)	Virtual DOM / Opt-In Vapor Mode (Direct DOM)	No Virtual DOM (Direct DOM Manipulation)
Rata-rata Ukuran Bundle	Tinggi (~148 KB gzipped avg SaaS)	Sedang (~94 KB gzipped avg SaaS)	Sangat Kecil (~31 KB gzipped avg SaaS)
Dukungan Tooling AI	Paling Dominan (v0, Lovable, Cursor, Prompt-to-UI)	Sangat Baik (General AI, Bolt.new)	Tumbuh Pesat (Svelte MCP Tool for AI Agents)
Kasus Penggunaan Terbaik	SaaS Enterprise, Aplikasi Skala Besar, AI Interfaces	SaaS Menengah, Produktivitas Cepat, Rapid Prototyping	Aplikasi Mobile-First, Bandwidth Terbatas, High Velocity

F. Pengujian Kualitas Kode & Analisis Statis (SonarQube Verification)

Seiring dengan meluasnya penggunaan asisten pemograman AI (AI Coding Agents) dalam pembuatan komponen UI, tim pengembang menghadapi tantangan yang dikenal sebagai verification bottleneck (SonarSource, 2026). Laporan riset menunjukkan bahwa 61% pengembang mengonfirmasi AI sering kali menghasilkan kode yang tampak benar namun tidak handal atau mengandung bug tersembunyi. Oleh karena itu, penerapan analisis statis otomatis menggunakan platform seperti SonarQube menjadi lapisan verifikasi wajib (verification layer) untuk menguji keandalan komponen, mencegah technical debt, dan memastikan kode UI memenuhi standar keandalan serta keamanan sebelum di-commit ke repositori utama (SonarSource, 2026).

4. Langkah-Langkah Praktikum Terpandu (Studi Kasus Proyek)

Pada praktikum ini, mahasiswa akan membangun komponen antarmuka kelola tugas proyek ("UNS Smart Task & Project Dashboard") menggunakan tiga pendekatan kerangka kerja UI utama (React 19, Vue 3, dan Svelte 5). Langkah terpandu ini bertujuan agar mahasiswa dapat membandingkan secara langsung sintaksis, reaktivitas, dan alur kerja komponen.

A. Langkah 1: Implementasi Komponen React 19 (React Compiler Auto-Memoization)

Buat berkas komponen **TaskDashboardReact19.jsx**. Perhatikan bagaimana React 19 mengelola state tugas tanpa memerlukan useMemo/useCallback eksplisit berkat React Compiler:

```
import React, { useState } from 'react';

// Komponen Card Tugas Produk (React 19)
export default function TaskDashboardReact19() {
  const [tasks, setTasks] = useState([
    { id: 1, title: 'Analisis SRS Proyek', category: 'D3TI-UNS', completed: true },
    { id: 2, title: 'Slicing UI dengan React 19', category: 'Front-End', completed: false },
    { id: 3, title: 'Pengujian SonarQube Verification', category: 'QA', completed: false }
  ]);
  const [filter, setFilter] = useState('all');
  const [newTaskTitle, setNewTaskTitle] = useState('');

  // Komputasi filter (Di-memoize otomatis oleh React Compiler)
  const filteredTasks = tasks.filter(task => {
    if (filter === 'completed') return task.completed;
    if (filter === 'active') return !task.completed;
    return true;
  });

  const handleAddTask = (e) => {
    e.preventDefault();
    if (!newTaskTitle.trim()) return;
    setTasks([...tasks, { id: Date.now(), title: newTaskTitle, category: 'Umum', completed: false }]);
    setNewTaskTitle('');
  };

  const toggleTask = (id) => {
    setTasks(tasks.map(t => t.id === id ? { ...t, completed: !t.completed } : t));
  };

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif', maxWidth: '600px', margin: 'auto' }}>
      <h2>📌 Task Dashboard (React 19 Auto-Compiler)</h2>
      <form onSubmit={handleAddTask} style={{ display: 'flex', gap: '10px', marginBottom: '15px' }}>
```

```

<input
  type="text"
  placeholder="Tambah tugas baru..."
  value={newTaskTitle}
  onChange={(e) => setNewTaskTitle(e.target.value)}
  style={{ flex: 1, padding: '8px', borderRadius: '4px', border: '1px solid #ccc' }}
/>
<button type="submit" style={{ padding: '8px 16px', background: '#1F4E79', color:
'#fff', border: 'none', borderRadius: '4px' }}>
  Tambah
</button>
</form>

<div style={{ marginBottom: '10px', display: 'flex', gap: '8px' }}>
  <button onClick={() => setFilter('all')} style={{ fontWeight: filter === 'all' ? 'bold'
: 'normal' }}>Semua</button>
  <button onClick={() => setFilter('active')} style={{ fontWeight: filter === 'active' ?
'bold' : 'normal' }}>Aktif</button>
  <button onClick={() => setFilter('completed')} style={{ fontWeight: filter ===
'completed' ? 'bold' : 'normal' }}>Selesai</button>
</div>

<ul style={{ listStyle: 'none', padding: 0 }}>
  {filteredTasks.map(task => (
    <li key={task.id} style={{ display: 'flex', justifyContent: 'space-between', padding:
'10px', background: '#f4f4f4', marginBottom: '6px', borderRadius: '4px' }}>
      <span style={{ textDecoration: task.completed ? 'line-through' : 'none'
}}>{task.title} ({task.category})</span>
      <button onClick={() => toggleTask(task.id)}>{task.completed ? 'Batal' :
'Selesai'}</button>
    </li>
  ))}
</ul>
</div>
);
}

```

11. Langkah 2: Implementasi Komponen Vue 3 (Composition API & Vapor Pattern)

Buat berkas komponen Single File Component Vue 3 **TaskDashboardVue3.vue** menggunakan Composition API (`<script setup>`):

```

<script setup>
import { ref, computed } from 'vue';

const tasks = ref([
  { id: 1, title: 'Analisis SRS Proyek', category: 'D3TI-UNS', completed: true },

```

```

    { id: 2, title: 'Slicing UI Vue 3 Vapor Mode', category: 'Front-End', completed: false },
    { id: 3, title: 'Pengujian SonarQube Verification', category: 'QA', completed: false }
  ]);

const filter = ref('all');
const newTaskTitle = ref('');

// Computed Property Reaktif
const filteredTasks = computed(() => {
  if (filter.value === 'completed') return tasks.value.filter(t => t.completed);
  if (filter.value === 'active') return tasks.value.filter(t => !t.completed);
  return tasks.value;
});

const handleAddTask = () => {
  if (!newTaskTitle.value.trim()) return;
  tasks.value.push({
    id: Date.now(),
    title: newTaskTitle.value,
    category: 'Umum',
    completed: false
  });
  newTaskTitle.value = '';
};

const toggleTask = (id) => {
  const task = tasks.value.find(t => t.id === id);
  if (task) task.completed = !task.completed;
};
</script>

<template>
  <div class="task-container">
    <h2>📌 Task Dashboard (Vue 3 Composition API)</h2>
    <form @submit.prevent="handleAddTask" class="form-group">
      <input v-model="newTaskTitle" type="text" placeholder="Tambah tugas baru..." />
      <button type="submit">Tambah</button>
    </form>

    <div class="filter-group">
      <button :class="{ active: filter === 'all' }" @click="filter = 'all'">Semua</button>
      <button :class="{ active: filter === 'active' }" @click="filter = 'active'">Aktif</button>
      <button :class="{ active: filter === 'completed' }" @click="filter =
'completed'">Selesai</button>
    </div>

    <ul class="task-list">
      <li v-for="task in filteredTasks" :key="task.id" class="task-item">

```

```

        <span :class="{ done: task.completed }">{{ task.title }} ({{ task.category }})</span>
        <button @click="toggleTask(task.id)">{{ task.completed ? 'Batal' : 'Selesai' }}</button>
      </li>
    </ul>
  </div>
</template>

<style scoped>
.task-container { max-width: 600px; margin: auto; padding: 20px; font-family: Arial, sans-serif; }
.form-group { display: flex; gap: 10px; margin-bottom: 15px; }
.form-group input { flex: 1; padding: 8px; border-radius: 4px; border: 1px solid #ccc; }
.form-group button { background-color: #42b883; color: white; border: none; padding: 8px 16px; border-radius: 4px; }
.filter-group { display: flex; gap: 8px; margin-bottom: 10px; }
.filter-group button.active { font-weight: bold; text-decoration: underline; }
.task-list { list-style: none; padding: 0; }
.task-item { display: flex; justify-between; padding: 10px; background: #f4f4f4; margin-bottom: 6px; border-radius: 4px; }
.done { text-decoration: line-through; }
</style>

```

B. Langkah 3: Implementasi Komponen Svelte 5 (Runes Reactivity \$state & \$derived)

Buat berkas komponen Svelte 5 **TaskDashboardSvelte5.svelte** menggunakan primitif reaktivitas Runes (`\$state` dan `\$derived`):

```

<script>
  // Svelte 5 Runes Reactivity Primitives
  let tasks = $state([
    { id: 1, title: 'Analisis SRS Proyek', category: 'D3TI-UNS', completed: true },
    { id: 2, title: 'Slicing UI Svelte 5 Runes', category: 'Front-End', completed: false },
    { id: 3, title: 'Pengujian SonarQube Verification', category: 'QA', completed: false }
  ]);

  let filter = $state('all');
  let newTaskTitle = $state('');

  // Derived State (Otomatis terbaru tanpa Virtual DOM)
  let filteredTasks = $derived(
    tasks.filter(t => {
      if (filter === 'completed') return t.completed;
      if (filter === 'active') return !t.completed;
      return true;
    })
  );
</script>

```

```

function handleAddTask(e) {
  e.preventDefault();
  if (!newTaskTitle.trim()) return;
  tasks.push({ id: Date.now(), title: newTaskTitle, category: 'Umum', completed: false });
  newTaskTitle = '';
}

function toggleTask(id) {
  const task = tasks.find(t => t.id === id);
  if (task) task.completed = !task.completed;
}
</script>

<div class="task-container">
  <h2>📅 Task Dashboard (Svelte 5 Runes Reactivity)</h2>
  <form onsubmit={handleAddTask} class="form-group">
    <input type="text" placeholder="Tambah tugas baru..." bind:value={newTaskTitle} />
    <button type="submit">Tambah</button>
  </form>

  <div class="filter-group">
    <button class:active={filter === 'all'} onclick={() => filter = 'all'}>Semua</button>
    <button class:active={filter === 'active'} onclick={() => filter = 'active'}>Aktif</button>
    <button class:active={filter === 'completed'} onclick={() => filter =
'completed'}>Selesai</button>
  </div>

  <ul class="task-list">
    {#each filteredTasks as task (task.id)}
      <li class="task-item">
        <span class:done={task.completed}>{task.title} ({task.category})</span>
        <button onclick={() => toggleTask(task.id)}>{task.completed ? 'Batal' :
'Seiasai'}</button>
      </li>
    {/each}
  </ul>
</div>

<style>
.task-container { max-width: 600px; margin: auto; padding: 20px; font-family: Arial, sans-
serif; }
.form-group { display: flex; gap: 10px; margin-bottom: 15px; }
.form-group input { flex: 1; padding: 8px; border-radius: 4px; border: 1px solid #ccc; }
.form-group button { background-color: #ff3e00; color: white; border: none; padding: 8px 16px;
border-radius: 4px; }
.filter-group { display: flex; gap: 8px; margin-bottom: 10px; }
.filter-group button.active { font-weight: bold; text-decoration: underline; }
.task-list { list-style: none; padding: 0; }

```

```
.task-item { display: flex; justify-between; padding: 10px; background: #f4f4f4; margin-bottom: 6px; border-radius: 4px; }
.done { text-decoration: line-through; }
</style>
```

5. Tugas Praktikum Mandiri (Case Project SRS Mahasiswa)

Dengan mengasumsikan setiap mahasiswa/kelompok telah memiliki dokumen Software Requirements Specification (SRS) dan ide proyek aplikasi masing-masing, mahasiswa diwajibkan membangun modul komponen antarmuka antarmuka pengguna (Front-End) menggunakan kerangka kerja pilihan (React 19, Vue 3, atau Svelte 5) berdasarkan ketentuan teknis berikut:

- **Penerjemahan Kebutuhan SRS ke Komponen UI:** Memetakan minimal 3 modul utama dari dokumen SRS proyek mahasiswa (misalnya: Module Dashboard, Module Form Entry, dan Module Data Table/List) menjadi struktur komponen UI yang terisolasi dan reusable.
- **Pengelolaan State & Reaktivitas Komponen:** Mengimplementasikan pengelolaan state komponen secara efisien sesuai paradigma framework yang dipilih (misalnya: useState/React Compiler pada React 19, ref/computed pada Vue 3, atau Runes \$state/\$derived pada Svelte 5).
- **Validasi Form & Handling Interaksi Sisi Klien:** Menyusun komponen formulir yang dilengkapi atribut validasi serta event handling interaktif untuk mencegah pengiriman data kosong dan memberikan umpan balik (feedback) visual kepada pengguna.
- **Simulasi Asinkronis Data API:** Mengintegrasikan simulasi pemanggilan data API asinkron (menggunakan Fetch API/Axios dengan async/await) untuk menampilkan indikator loading, error boundary/state, dan rendering data dinamis.
- **Pengukuran Performa Komponen:** Melakukan analisis sederhana terhadap ukuran bundle dan waktu render antarmuka untuk memastikan komponen tidak mengalami masalah re-render yang berlebihan.
- **Dokumen Matriks Pemetaan SRS vs UI Component:** Menyusun laporan singkat (PDF 1–2 halaman) yang memuat tabel pemetaan antara ID Kebutuhan SRS dengan nama file komponen UI yang telah dibuat.
- **Struktur Berkas Proyek & Verifikasi Kode:** Mengorganisasikan repositori proyek secara rapi dan memastikan kode bebas dari kesalahan sintaksis atau bug kritis berdasarkan standar analisis statis.

Rubrik Penilaian Tugas Praktikum (Skala 100 Poin)

Kriteria Penilaian	Bobot Poin	Indikator Penilaian Utama
Kesesuaian Komponen vs SRS	25 Poin	Komponen UI dibuat tepat mewakili modul kebutuhan fungsional dokumen SRS mahasiswa.

Penerapan Reaktivitas Framework	25 Poin	Penggunaan state/runes/computed property secara tepat dan efisien tanpa merusak siklus render.
Interaktivitas & Validasi Form	20 Poin	Formulir berfungsi baik, memiliki validasi masukan, dan memberikan umpan balik pengguna yang jelas.
Simulasi Asinkron & Loading State	15 Poin	Berhasil mensimulasikan pemanggilan API asinkron beserta indikator loading dan penanganan error.
Kerapihan Kode & Dokumentasi	15 Poin	Struktur folder rapi, penamaan berkas konsisten, dan melampirkan matriks pemetaan SRS vs UI.